

Process Mining Software Repositories from Student Projects in an Undergraduate Software Engineering Course

Megha Mittal
megha1243@iiitd.ac.in

Ashish Sureka
ashish@iiitd.ac.in

Indraprastha Institute of Information Technology, Delhi (IIITD)
New Delhi, India

ABSTRACT

An undergraduate level Software Engineering courses generally consists of a team-based semester long project and emphasizes on both technical and managerial skills. Software Engineering is a practice-oriented and applied discipline and hence there is an emphasis on hands-on development, process, usage of tools in addition to theory and basic concepts. We present an approach for mining the process data (process mining) from software repositories archiving data generated as a result of constructing software by student teams in an educational setting. We present an application of mining three software repositories: team wiki (used during requirement engineering), version control system (development and maintenance) and issue tracking system (corrective and adaptive maintenance) in the context of an undergraduate Software Engineering course. We propose visualizations, metrics and algorithms to provide an insight into practices and procedures followed during various phases of a software development life-cycle. The proposed visualizations and metrics (learning analytics) provide a multi-faceted view to the instructor serving as a feedback tool on development process and quality by students. We mine the event logs produced by software repositories and derive insights such as degree of individual contributions in a team, quality of commit messages, intensity and consistency of commit activities, bug fixing process trend and quality, component and developer entropy, process compliance and verification. We present our empirical analysis on a software repository dataset consisting of 19 teams of 5 members each and discuss challenges, limitations and recommendations.

Categories and Subject Descriptors

D.2.8 [Software Engineering]: Metrics; K.3.2 [Computers and Education]: Computer and Information Science Education; H.2.8 [Database Applications]: Data Mining

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

ICSE Companion '14, May 31 – June 7, 2014, Hyderabad, India
Copyright 2014 ACM 978-1-4503-2768-8/14/05...\$15.00
<http://dx.doi.org/10.1145/2591062.2591152>

General Terms

Algorithms, Experimentation, Measurement

Keywords

Mining Software Repositories, Process Mining, Education Data Mining, Learning Analytic, Software Engineering Education

1. RESEARCH MOTIVATION AND AIM

Software Engineering (SE) is a practice-oriented and applied discipline and software development processes, team-work, project management and exposure to popular SE tools are important learning objectives in addition to fundamental technical concepts in SE courses [2][4][5][8][9][11][14][15]. SE courses generally consists of a team-based semester-long project giving an opportunity for students to apply the theory and principles learnt during lectures. Student teams follow a development process to deliver a software product. Several artifacts such as software requirement specification document (SRS, design document, project plans, test plans and source code) are produced at various milestones during product development. A course instructor can easily assess and provide feedback on product and deliverables such as SRS as it is visible but providing feedback on process and team-work is not straightforward as it is not explicitly visible unlike a product. The study presented in this paper is motivated by the need to provide an effective mechanism for SE course instructors to gain visibility and insights on software development processes followed by student teams and provide appropriate feedback on process improvement. Our motivation is to develop tools and techniques for solving problems encountered by SE course instructors.

Several SE tools such as online Wiki based collaborative document editor, Version Control Systems (VCS), Issue Tracking System (ITS), project management and testing tools facilitating team-work and product development are taught during the course. Tools such as ITS, VCS and Wiki generate an event log of activities performed by developers. For example, VCS records who, when and what of a software change. Wiki event log records who (actor) made what change and when (timestamp). Event logs generated by such Process Aware Information Systems (PAIS) can be process mined for discovering run-time process map, imperfections, inefficiencies and interesting patterns [15]. Process mining event logs generated from PAIS used by student-teams in an

Table 1: Seven Closely Related Work Arranged in Chronological Order

Study	University	Repository	Objective
Liu 2004 [8]	University of Alberta	CVS	Studying correlation between grades and nature of collaboration.
Mierle 2005 [9]	University of Toronto	CVS	Mining statistical patterns or predictors of performance.
Glassy 2006 [2]	University of Montana	SVN	Mining problematic patterns, steadiness of progress and quality of commit messages
Kay 2006 [5]	University of Sydney	Wiki, ITS and SVN	Frequent pattern characterizing aspects of team-work.
Jones 2010 [4]	Bloomsburg University	SVN	Aid in determining the accomplishments of each individual in a group programming project.
Poncin 2011 [11]	Eindhoven University of Technology	SVN, Mails, Wiki Articles Logs	Identifying developer roles, development models and prototype reuse
Robles 2013 [14]	Universidad Rey Juan Carlos	GIT VCS	Gathering software analytics data from programming assignments

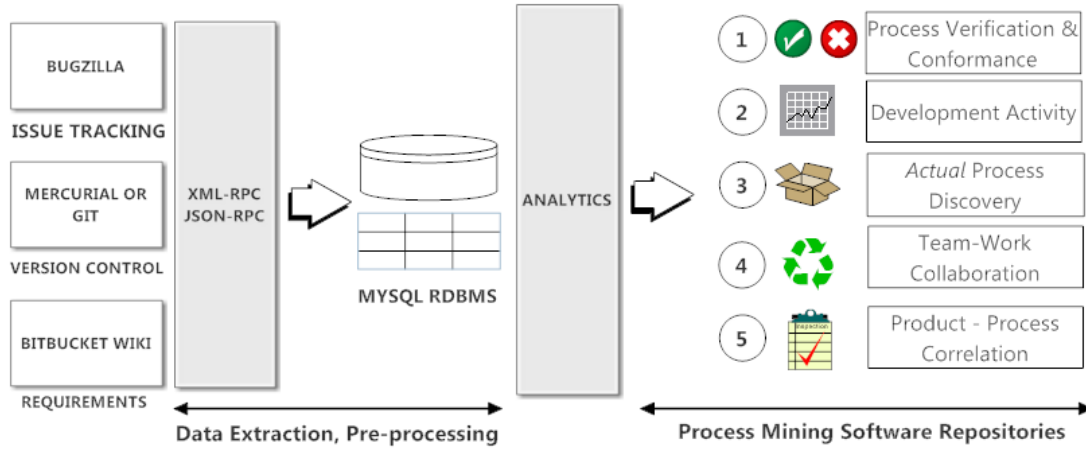


Figure 1: Research Framework

education setting can provide feedback and actionable information to the course instructor. The specific research aim of the work presented in this paper is the following:

1. To investigate the application of process mining event log data generated from Wiki based collaborative document editor, Version Control Systems (VCS) and Issue Tracking System (ITS) in the context of an undergraduate SE course.
2. To define new process quality metrics and visualization and examine the application of existing process quality metrics for the purpose of understanding software development process by student teams in an educational setting
3. To conduct a case-study and empirical analysis of process mining software repositories and activity logs in a semester-long undergraduate level SE course and present our (teaching team) experiences, effectiveness of the proposed approach as well as challenges encountered.

2. RELATED WORK AND RESEARCH CONTRIBUTIONS

In this Section, we present closely related work to the study presented in this paper and present novel research

contributions in context to existing work. We characterize closely related work as formal studies on mining software repositories of student team-based projects within the context of university level Software Engineering course.

Table 1 presents the outcome of our literature survey on closely related work. We categorize 7 chronologically sorted papers (refer to Table 1) based on three attributes: software repositories used in the study, name of the University and objective of the research study. Our literature survey reveals that mining software repositories in an educational setting is an area that has attracted the attention of several instructors teaching university level SE courses and the study presented in this paper aims to add the body of knowledge on the topic. Table 1 shows that mining VCS is the most common application and mining other software repositories like Wiki based content management system and issue tracking system is relatively unexplored. In context to existing work, the novel research contributions of this paper are following

1. We present an application of process mining Wiki, VCS and ITS event logs in a SE course setting consisting of 19 teams of 5 members each. While there are several studies on mining VCS, analyzing wiki and ITS activity event logs from a process mining perspective is a fresh perspective presented in this paper.

Table 2: Experimental Dataset Details

Field	Value	Field	Value
Total Students	99	VCS : Total Commits	925
Total Teams	19	VCS : Average Commits	50
Date of First Wiki Activity	21-08-2013	Date of First ITS Activity	04-09-2013
Date of Last Wiki Activity	10-10-2013	Date of Last ITS Activity	17-10-2013
Wiki : Total Commits	1167	ITS : Total Bug Reports	482
Wiki : Average Commits	60	ITS : Average Bugs Reported	26
Date of First VCS Activity	23-08-2013	ITS : Maximum Bugs Resolved	404
Date of Last VCS Activity	17-10-2013	ITS : Average Bugs Resolved	22

2. We present several visualizations and metrics providing feedback on various process aspects such as: workload distribution between team members, regularity in contributions from start till the deadline, quality of commit messages, component and developer entropy, quality of efficiency of bug fixing process.
3. We conduct a case-study and apply the proposed approach on software repository dataset generated from 19 student projects in an undergraduate level SE course. We present an experience report, results and challenges encountered.

3. RESEARCH METHODOLOGY AND EXPERIMENTAL DATASET

We propose a research framework (called Mimansa) illustrated in Figure 1 and conduct experiments on the dataset described in Table 2. As shown in Figure 1, the architecture of Mimansa consists of multiple components and phases. We mine three process aware information systems (PAIS): Bugzilla¹ issue tracking system, Mercurial² and Git³ version control system and Bitbucket⁴ Wiki. We extract raw data from PAISs using XML-RPC and JSON-RPC API calls and store the data in a MySQL database. The extracted data is then pre-processed and transformed to a format which is suitable to a mining task (for a given objective). Fields which are common across event logs from all the three systems are: activity timestamp, actor and the action. We frame several questions (actionable information for the instructor such as the degree of contribution from team members towards a common goal) and define process quality metrics and visualizations which are addressed in the analytics phase of the research framework. As shown in Figure 1, the outcome of process mining is multi-dimensional and derived by mining data from multiple PAIS giving visibility to the instructor on the development process in addition to the product.

Software Engineering (CSE 300) is a 4 credit core course (two classes of 1.5 hours each in one week with a total of 26 classes over 4 months) offered during the third year (5th semester) of Bachelor of Technology (abbreviated as B.Tech) in Computer Science and Engineering (CSE) program at Indraprastha Institute of Information Technology (IIIT Delhi, a state university in India). The experience report is based

on the SE course taught in Monsoon 2013 semester. We created 19 teams of 5 members each (except few teams which had 6 members). Table 2 describes the experimental dataset collected over a period of 8 weeks. The start date for the experimental dataset is the begin date of wiki activity (21-August-2013) until the last date of ITS activity (17-October-2013). As shown in Table 2, we observe a total of 1167 commits in Wiki, 925 commits in VCS and 482 issues in Bugzilla. We notice that 404 out of 482 issues reported to ITS are resolved. We defined several project deliverables during the course. The first project deliverable was on creating 25 – 30 user stories and a release plan (using team wiki). One of the project deliverables was to conduct functional and non-functional testing of the software and use ITS for managing and tracking defect reports as well as feature enhancement requests.

4. TEAM WIKI

We taught basic principles and practices of Extreme Programming (XP) during the course and one of the project deliverables was on writing user stories, product backlog and release plan according to Extreme Programming methodology [10]. Agile development processes are intended to support early and quick production of working code by structuring the development into small release cycles and focussing on continual interaction between developers and customers. User stories are one of the primary development artifact of this approach and are used as the basis for defining the functions a system must provide and to facilitate requirements management [12]. It captures the 'who', 'what' and 'why' of a requirement in a simple, concise way specifying the needs and expectations of the user. Team Wiki's were used by the students for the documentation of user stories. A wiki is a collaborative tool that supports software development and maintenance process of a team project.

Whatever changes are done to the information in a wiki leaves its traces in an event log. A wiki event log represents the activity of developers during requirement specification documentations. Figure 2 is a snapshot of Bitbucket Wiki feature in which we mask the actual names of the authors and label various data fields. From this log, we obtain information like a timestamp corresponding to an activity, summary of commit and its author. Using this information we analyze the process and practise followed by students during the software development.

4.1 Quality of Commit Messages

Wiki commit messages are like documentation and writing quality and well-formed commit message is a process attribute and not a product (a document containing user-

¹<http://www.bugzilla.org/>

²<http://mercurial.selenic.com/>

³<http://git-scm.com/>

⁴<https://bitbucket.org/>

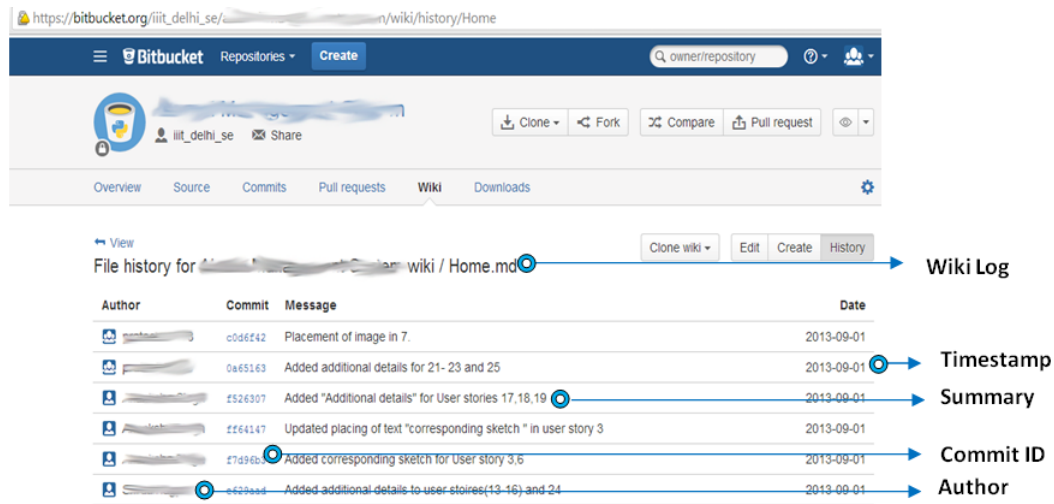


Figure 2: A Snapshot of Wiki Event Log

Table 3: Categorization of Commit Messages into Three Levels of Quality

Quality	Example	Remarks
Poor	1. Edited online. 2. Wrote 4 user stories. 3. Minor fixes here and there. 4. User Story-32	Vague, not specific, only 2-3 terms, too general.
Average	1. Changes to story 8,9,10,14. 2. Edited some user stories(Priority heading removed). 3. Edited 2nd user story. 4. Image issues of 20.	Specific but still not covering all (rationale, why and what) relevant details.
Good	1. Re added costs for 9-12,got removed due to commit conflict. 2. FAQ page addition for easier usage of the application. 3. Added Title, Priority and Cost for User Stories 17,19. 4. Release of user story 1 and 2 added.	Short, concise, specific and explain the rationale, why and what.

stories) attribute. We emphasized in class on writing good quality commit messages (making a habit of writing high quality commit messages) as it results in good documentation, gives a better insight and helps in debugging or answering questions. We conducted a qualitative analysis of the commit messages to check if they are clear and concise, explains the rationale and mention the concepts like how and why this change was committed. Following is the process quality question useful to the instructor from a pedagogy perspective:

RQ1 : *To what extent guidelines on commit message quality are followed and what are the specific gaps.*

We categorize the quality of commit messages into three broad classes: Good, Average and Poor. Table 3 shows examples of few commit messages in each category from student projects. We observe commit messages like "Edited some user stories" in which it is not clear which user story is edited. We notice commit messages like "Edited online", "Minor fixes here and there" and "Wrote 4 user stories" which are vague and poor quality. We also observe several commit messages (refer to Table 3) like "Added title, priority and cost for user stories 17 and 19" which is short, concise and specific. Quality of commit messages was a parameter for

grading the project deliverable and the grade was not only based on product quality but also process quality.

4.2 Consistency in Commit Activity

Software Engineering is a 15 weeks long course with a project that should evolve in an incremental and iterative fashion(reflecting a real world development environment). Analysing the commit activity of a team wiki helps in understanding the nature of evolution of a project during the documentation of user stories. We define the commit activity of a team as the frequency of commits across timeline. A consistent pattern in the frequency of commits by a team reflects an efficient pattern in the evolution of a project. Analyzing the frequency in commit activity will help the instructor in answering the following research question:

RQ2 : *To what extent the process followed by the students during the requirement documentation of their projects is incremental and iterative.*

Figure 3 shows different commit activity patterns of the teams. We notice an incremental pattern in the frequency of commit activity for team like "Facility" and at the same time a non-incremental pattern can be noticed for team like "Hostel". Reviewing the commit activity of teams across timeline unfolds another important aspect of an educational

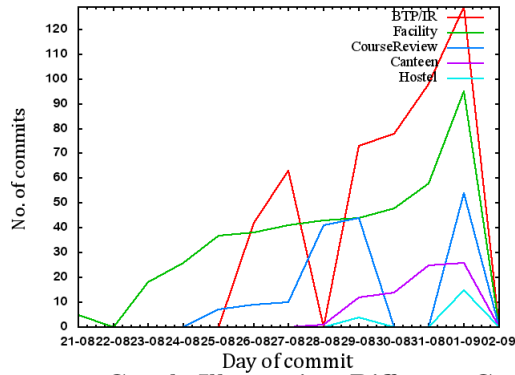


Figure 3: Graph Illustrating Different Commit Activity (Number of Commits Across Time) Patterns

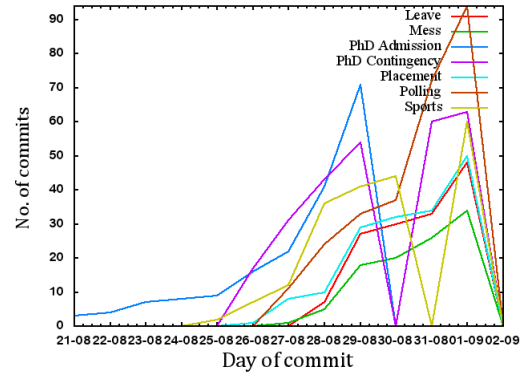


Figure 4: Graph Illustrating Significant Increase in Number of Commits Close to the Deadline

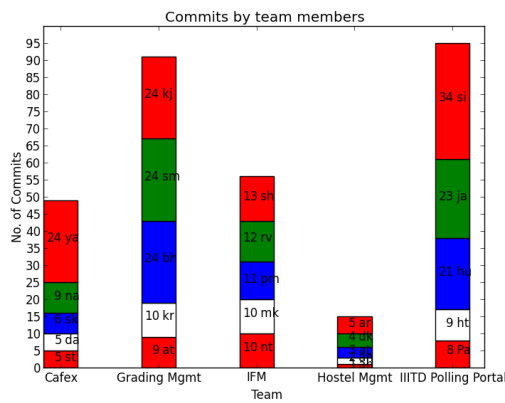


Figure 5: Contribution of Each Member of a Team in a Wiki

environment and that is reviewing the amount of work done by students near to the deadline. From Figure 4 we observe that most of the teams follow a non-uniform process and work more frequently near to the deadline. Figure 4 reveals that team “Polling” contributes 35% of the total work on the last day of the deadline. While we see an increase in activity in last 2 days, there is work in the start and middle as well, as can be seen in projects “PhDAdmission” and “PhDContingency”. Project work for team “PhDAdmission” started early and finished off well before the deadline. Figure 3 and Figure 4 shows multiple patterns observed from wiki activities of different teams, though we do not plot patterns for all the projects and represent the activity of only a few projects.

4.3 Contribution of Members in a Team Wiki

Software Development Process inherently requires team work with an equal distribution of work load across all the members of a team for the development of a healthy product. We encouraged students to put an emphasis on uniform team work with an aim to develop the quality of team spirit amongst them. Student team projects are a hallmark of undergraduate Software Engineering courses and programs [3][13]. We look into the contribution of team members in a team wiki to provide an insight to the instructor from the

following point of view:

RQ3 : *To what extent the work load distribution in a team was uniform or skewed.*

A stacked chart is used as shown in Figure 5 to represent the distribution of work across the members of teams. Figure 5 shows different levels of work load distribution across different members of teams. We observe that team members can be divided into 3 broad categories namely: Out-performers as shown by team “Cafex” with member “ya” contributing 45-50% of the total work, Equal performers, team “IFM” is an example of this category in which each member contributes 20-25% each and below expectation performers like in team “IIITD Polling Portal” where member “Pa” is contributing only 8% of the total work. Such a differentiation in students further benefits the instructor in adopting a fair grading policy.

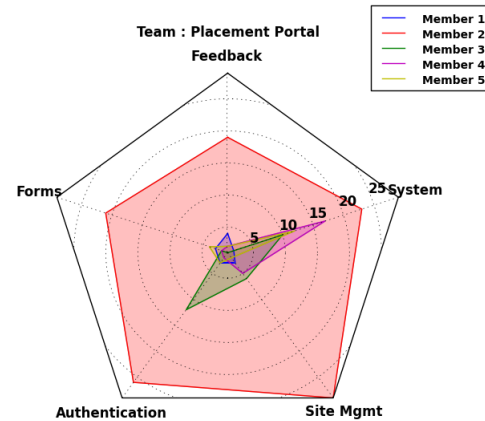


Figure 6: Radar Chart Illustrating Developers Contribution Across Components

5. VERSION CONTROL SYSTEM

Version Control Systems(VCS) are essential tools in software development of a project. In an educational setting, use of VCS is not only beneficial for students as it prepares them for real life situations but has several potential ben-

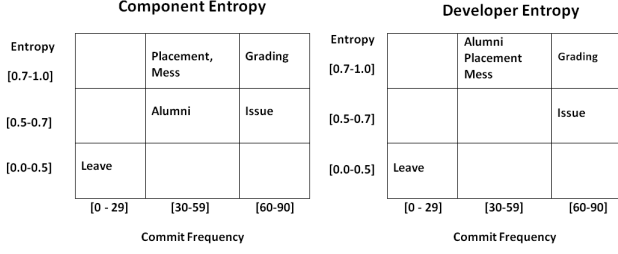


Figure 7: Component and Developer Entropy Graphs Showing the Position of 6 Teams

efits for the instructor as well. It can be used as a tool to monitor or visualize team and individual contribution.

The event log of a VCS is the most important documentation for any project. We analyze the event log of the projects to get an insight on process followed by student teams during the development phase.

5.1 Component and Developer Entropy

SE courses generally include a team-based semester-long project where student teams follow a development process to deliver a software product. Several artifacts such as software requirement specification document (SRS), design document, project plans and source code are produced at various milestones during product development. A course instructor can easily assess and provide feedback on product and deliverables such as SRS as it is visible but providing feedback on process and team-work is not straightforward as it is not explicitly visible unlike a product. In student projects, analysing the contribution of individual developers in a team not only helps the instructor in understanding the quality of team work involved but also helps in fair grading. Every project can be divided into several modules or components which require a collaborative effort from all members of a team to yield an efficient product. The students were asked to follow a modular structure for the development of their project. We study the uniformity in the contribution of developers across all the components of a project to facilitate instructor understanding the following:

RQ4 : *To what extent the contribution of work amongst the developers of a team across various components was uniform.*

We define that team work in a project will be perfect if the contribution of all the members of the team is uniformly distributed across various components of the developed software.

We examine the VCS event logs to determine the contribution of each developer across all the components of a project. A Radar Chart as shown in Figure 6 is used to represent the contribution of all the team members across various components. Each axis represents a component of the software and each colour represents a developer. An equal distribution of a colour along all axes represents equal work by a developer. A skewed distribution of a colour shows an expertise area of a developer but shows lack of participation in all the components of the software.

Figure 6 shows the contribution of developers of a team within a component and across the components. It shows

participation of member2 equally spread to all the components whereas a peak in the contribution of member3 and member4 in components “Authentication” and “Systems” respectively shows their expertise in the respective areas but lack of contribution to other parts of the software. It also helps in spotting the least contributors of a project as well like “member1” who worked least in the entire team.

We define two metrics called Component Entropy (refer to Equation 1) and Developer Entropy (refer to Equation 2) to quantify two types of distribution (developer-component and component-developer) serving as an indicator of extent of co-development at component level. Lal et al.[7] apply the concept of Entropy for calculating component and bug-reporter entropy across seven different types of defect reports and Khomh et al.[6] present an application of bug triaging by computing crash-entropy (calculating entropy based on crash-types across users).

$$H(d|c) = - \sum_{j=1}^{j=m} \sum_{i=1}^{i=n} p_{(d=i|c=j)} * \log_n(p_{(d=i|c=j)}) \quad (1)$$

$$H(c|d) = - \sum_{i=1}^{i=n} \sum_{j=1}^{j=m} p_{(c=j|d=i)} * \log_m(p_{(c=j|d=i)}) \quad (2)$$

We instructed the student teams to have a modular design of their system consisting of components or modules. A component consists of a collection of files and components are inter-dependent. If two developers modify same files or files within the same component then we refer it to as sharing, co-development or collaboration at the component level.

Let us say a team of n members is developing a system consisting of m components. Developer-component entropy (Equation 2) quantifies the distribution of n developers across m components. Inner sum calculates the individual entropy of each developer which is then averaged over all the developers. Similarly, Component-developer entropy (Equation 1) refers to the distribution of each of the m component across n developers. Here, Inner Summation represents the entropy value of individual components where a high value indicates an equal contribution from all the developers in this component. “p” refers to as the conditional probability where in Equation 1, term $p_{(d=i|c=j)}$ indicates probability of number of commits by developer i given component j and in Equation 2, term $p_{(c=j|d=i)}$ indicates probability of number of commits in component j given developer i . The entropy value can vary from a minimal of 0 to a maximum of 1. The entropy is maximal (one extreme end) if there is a perfectly uniform distribution (every component receives equal contribution by all the team members). On the other extreme end, entropy value is minimal if there is no co-development or collaboration at component-level.

Figure 6 displays the level of collaboration in the development of project “Placement”. The developer entropy for member2 is 0.99 showing his equal contribution to all the components. The overall Developer-component entropy for “Placement” is 0.87. Similarly, we calculate the component entropy for component “Feedback” as 0.64 reflecting unequal contribution of all the team members in this part of the project, on the other hand the component entropy for component “System” is 0.88. The overall Component-developer entropy is coming out to be 0.72. Figure 7 represents the

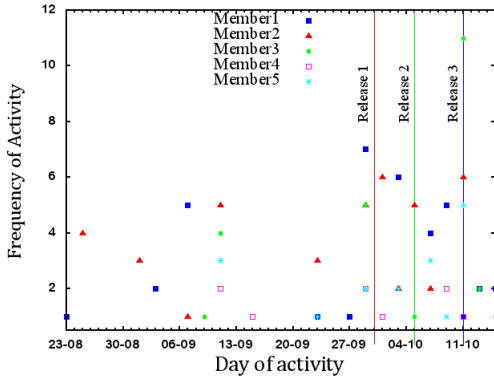


Figure 8: Scatter Plot Representing the Commit Activity with Release Dates

component and developer entropy and the frequency of commits for 6 teams. We notice that for team “Leave”, component and developer entropy value lie in a low value region of 0-0.5 representing poor team work whereas high entropy value for teams like “Grading”, “Alumni”, “Placement” and “Mess” indicate collaborative development of the projects. The entropy values obtained from the above mentioned metrics reflect the diversity in the contribution of work in a project and not the intensity. One of the limitations of this metric is that if a developer works less but still shows uniformity in his contribution to all the components, then we get a high value of developer entropy. The same applies to the component entropy as well. One way to extend this metric and remove this limitation is to add weights or a bias during the calculation of the entropy values.

5.2 Effects of Milestones on Commit Behaviour

The development process of a software is an incremental process that can take multiple iterations offering a series of releases with additive functionality. Analyzing the development activity of projects near release dates helps in understanding the change in the behaviour of students with deadlines. We focussed on the commits that involve source code contributions of different developers of a project and their commit frequency to understand their behaviour near pre-set milestones or release plans to help instructor understand the following aspect:

RQ5 : *To what extent did various milestones or release plans effected the activity of developers of a project.*

Figure 8 shows the commit frequency of different developers of team “CourseReview”. Each colour represents a different developer and its position in the graph represents the amount of work done in terms of number of commits. Release dates associated with this project are represented by the vertical lines. An increase in commit frequency shows an increase in the development activity of a project. Figure 8 represents an increase in the density of commit activity near the release dates with an increase in number of commits by majority of the developers. From this we infer that developers become more active near to the release of a deliverable and their work is not uniformly distributed throughout the development process. We also notice that member2 shows a consistent behavior with decent commit activity throughout the timeline indicating a member of the team who takes timely actions in the development process of the project.

6. ISSUE TRACKING SYSTEM

Issue Tracking Systems such as Bugzilla are Process Aware Information Systems used during software development and maintenance for issue (feature enhancement requests and defects) tracking and management of a project. Software maintenance and defect-fixing process is a collaborative activity involving several roles such as bug reporter, bug fixer and project developers. An ITS produces an event log that contains the information about the steps followed during bug-fixing process. Event log contains information like who reported a bug and when and changes in resolution and status of a bug with their respective timestamps.

Process data archived in an ITS as an event log can be mined to track the bug-resolving process of a project and can also be used for contribution and performance assesment of individuals as a bug reporter and fixer. In the Software Engineering course, Bugzilla was introduced as an ITS to be used for the defect tracking and maintenance of their products. Each team project was allocated a testing team that was responsible for testing their projects. Team members were also responsible for testing their own projects apart from the testing of projects allocated to them. In the following sections we provide visualizations and metrics to accurately and reliably measure the contribution of student teams as developers and testers during the software maintenance and bug-fixing process.

6.1 Bugs Opening Trend, Closing Trend and Continuity

Fractalanci et al.[1] present a method to measure performance characteristics such as continuity and efficiency of the bug fixing process during software maintenance [1]. They define two performance indicators (bug opening and closing trend) reflecting the characteristics and quality of bug fixing process. We mine the issue tracking system of student projects and apply the performance indicators proposed by Fractalanci et al. to measure the bug fixing performance efficiency [1]. Fractalanci et al. define bug opening trend as the cumulated number of opened bugs over time and closing trends as the cumulated number of bugs that are resolved and closed over time [1].

We plot the opening and closing trend for all projects on a graph and investigate the similarities and differences in their characteristics. In Figure 9, x-axis represents the timeline across which bugs are reported and resolved and y-axis shows the bugs count associated with each date. Figure 9 displays the opening and closing trend for three projects exhibiting different characteristics. At any instant of time, the difference between the two curves (interval) can be computed to identify the number of bugs which are open at that instant of time. We notice that the debugging process is of high quality for the graph in Figure 9(c) as there is no uncontrolled growth of unresolved bugs (the curve for the closing trend grows nearly as fast or has the same slope as the curve for the opening trend) across all time instants. However, we notice in Figure 9(a) that the growth of unresolved bugs increase with time and that they are resolved only when the deadline approaches. This shows an inefficient growth and process of software testing. Figure 9(b) shows a team with an average trend of bug fixing process.

We propose a metric called as “Bug Fixing Score” as shown in Equation 3 to quantify the bugs opening and closing trend

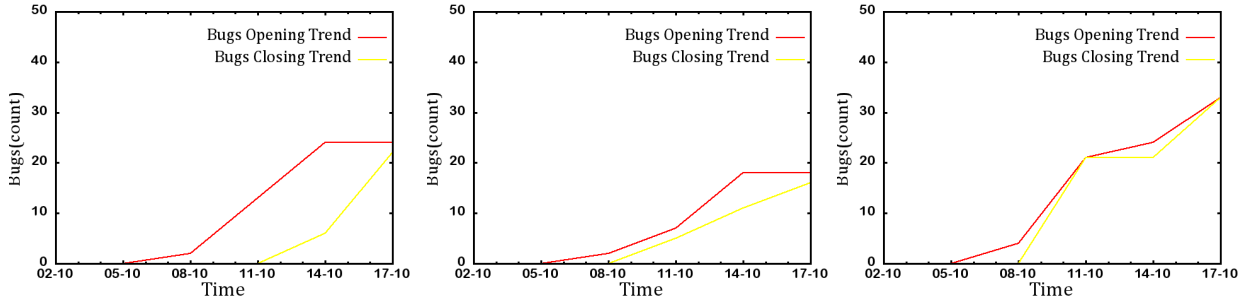


Figure 9: Bugs Opening And Closing Trends with 3 Different Behaviour as in Subfigures (a), (b) and (c)

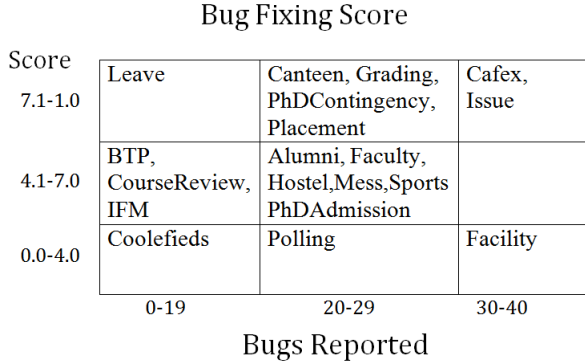


Figure 10: Bugs Fixing Score Graph Showing the Position of 19 Teams

of a project. The metric can serve as an indicator of the bug fixing performance efficiency.

$$BugFixingScore = (1/(n - i)) * \sum_{i=1}^n \log(BO_i - BC_i) / \log(BO_i) \quad (3)$$

where, BO = Bugs opened till time instant i, BC = Bugs closed till time instant i, “i” is the first time instant when BO>0 and “n” is the maximum recorded time instant. The limit for inner sum goes from “i” to “n”, that is, from the time instant the first time bugs opening starts till the maximum recorded time instant. Score of a project can vary from a minimal value of 0 to a maximal value of 1 where value 0 indicates a perfectly efficient behaviour where the bug fixing process is as fast as the bug opening process. On the other extreme end, score value 1 indicates an inefficient behaviour where there is no bug fixing throughout the process though number of bugs opened continue to increase over time. The Bug Fixing Score helps the instructor in answering the following research question:

RQ6 : *To what extent the bug fixing performance of a project was efficient.*

We calculate the score of all projects as shown in Figure 10 and place them according to the value of their score with respect to number of bugs reported. In Figure 10, x-axis represents the count of bugs reported and y-axis shows bug fixing score divided over 3 intervals. We conclude that 50% of the projects exhibited a normal behaviour in bug-fixing process with a score value in range 4.1-7.0 whereas only 16% of the teams showed an efficient behaviour in the bug-fixing process lying in a range between 0.0-4.0. Rest

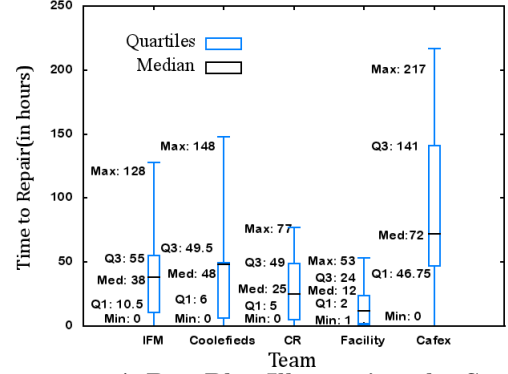


Figure 11: A Box Plot Illustrating the Statistics for Time to Repair of Bugs Reported

of the teams showed an inefficient behavior in the bug fixing process. In Figure 10, bottom right corner represents a team which reported considerable number of bugs with an efficient value of bug fixing score lying in the range of 0.0-4.0. From this graph, one can easily comprehend and visualize the performance of various teams during the testing and software maintenance phase of projects. The instructor can take benefit of this metric during grading of projects by understanding the efficiency of student teams during the bug reporting and resolving process of their respective projects.

6.2 Mean Time to Repair

Time taken to repair a bug (TTR) is a measure that helps in determining the efficiency of the developers during the testing phase of a project. We define the time to repair a bug as follows:

$$TTR(b)(inHours) = date_{closedb} - date_{openedb} \quad (4)$$

Analysing the repairing activity obtained from the time taken by developers to resolve bugs helps in quantifying the devotion and importance given by the development team to the testing and maintenance process of their projects and serves as a measure of the seriousness of developers to the issues faced by their customers. The statistics obtained from bugs repairing time of student teams helps the instructor to comment on the following research question:

RQ7 : *What are the variations in the time devoted and importance given by the development teams to the repairing activity of bugs.*

We use a box plot as shown in Figure 11 to represent the statistics related to the TTR of bugs for different teams.

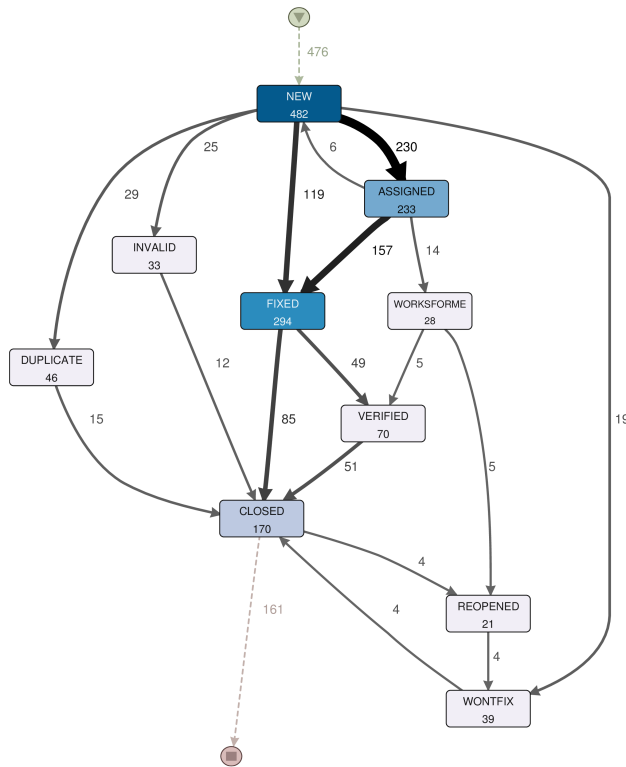


Figure 12: Run-Time Bug Life-Cycle Process Map

Figure 11 shows the quartiles and median values for 5 teams. A high value of “Q3” indicates that bugs in this project were left unnoticed or no importance were given to the bugs resolving process thus shedding light on the irresponsible behaviour of the development team towards importance of testing process. From Figure 11 high value of Q3 for team “cafe” indicates an inefficient behaviour of developers of this team towards the bug resolving process. On the other hand we observe that most of the teams are showing an efficient behaviour in bug resolving process taking time between 5 to 60 hours, that is, the bugs are getting resolved within a timespan of 2 days from their reported time. Resolving bugs in sufficient time not only reflects the sincerity of the development team towards the maintenance process of their product but also motivates them to avoid such situations in their future releases.

6.3 Component vs Priority in a Project

A software project can be divided into various sub-modules or components developed by the collaborated efforts of the developers of a project. Rigorous testing and inspection is required in each part of a project for successful and reliable delivery of the product. During the testing phase of a software, all the components are tested and bugs with different priority are reported according to the importance of their functionality. A component with majority of the bugs in high priority region not only indicates its importance from the users point of view and urgency to correct them but also shows an unsuccessful attempt of the development team in

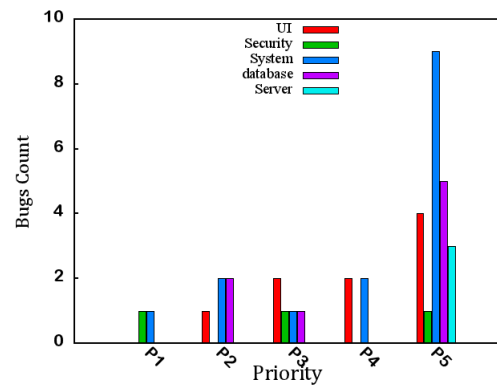


Figure 13: A Histogram Showing the Distribution of Bugs Priority Across Various Components of a Project

serving the main functionality of this component. For the student projects, such a review will help the instructor in identifying:

RQ8 : *What are the components of a project that turned out to be weak and less functional during the testing phase and what are the reasons for such a behavior.*

Figure 13 is a histogram which shows the distribution of bugs with different priority in various components of a project. We notice that maximum number of bugs are reported in component “system” that shows its importance in the project and indicates lack of expertise in serving the main functionality of this product. On the other hand, component like “security” with less number of bug show the expertise and successful attempt of the project team in serving the need of the user. In an educational setting, such a graph also helps the instructor during the viva of the student projects as the instructor can ask the reasons for the failure of a particular component, who was responsible for the development of this part and what steps were taken to avoid such situations in the future releases of the project.

6.4 Process Discovery

Event log archived in ITS stores information like what was changed as in status or the resolution of a bug and the timestamp associated with the change. We mined the event logs collected from the bug fixing activity of student projects and mined it to get an insight into the process followed by student teams during the software testing and maintenance phase. In the Software Engineering course, the instructor emphasised student teams to follow design time process model of a bug lifecycle during bug fixing process. We review the process models of the bug fixing activity in projects to give an insight to the instructor from the following perspective:

RQ9 : *To what extent the runtime process models of student teams conform with design time process model.*

There are many process mining tools like ProM(Open source) and Disco(commercial)⁵ used to obtain process models of softwares. We used Disco⁶ as a tool to obtain process map and other statistical information for testing process of

⁵Disco is a proprietary tool for which we availed academic license and used for our analysis.

⁶[http:// fluxicon.com/disco/](http://fluxicon.com/disco/)

student projects. Disco miner is based on the proven framework of the Fuzzy Miner with a completely new set of process metrics and modelling strategies⁷. It is used to discover the runtime process from the actual event log generated during the progression of a bug. Bug ID is selected as case (for process map generation in Disco) to associate all activities pertaining to the same bug ID and hence, we can visualize the lifecycle of a bug. Preprocessed data is imported into Disco which is used to discover the runtime or actual process from the event log generated during the progression of a bug. We provide the data collected from event logs to Disco with 3 types of information:

Bug ID as the Case ID, Status field or resolution field as activity, and event log timestamp as timestamp. Figure 12 reveals the bugs activity process of the student projects. Each node represents an activity and an edge between two nodes represents a transition from one activity to another during the bug lifecycle. Total reported bugs are 476 which travel through 10 different stages in their lifecycle represented by 10 nodes. Label of each edge indicates the absolute frequency of transition, shade and thickness corresponds to the frequency with more frequent being dark and less frequent as light. We notice that 29% of bugs were marked as “closed” right after they were “fixed” without getting “verified”. Around 7% and 5% of the bugs were reported as “invalid” and “wontfix” respectively. Process Map shown in Figure 12 was further analysed to review activity frequency and transition frequency. We observe that once a bug is reported as “new” the most frequent event that it passes through is “assigned” followed by “fixed”. We infer from the analysed process map that *New* $\rightarrow$ *Assigned*, *Assigned* $\rightarrow$ *Fixed* and *New* $\rightarrow$ *Fixed* are the most frequent transitions.

7. CONCLUSIONS

Mining activity logs generated by process aware information systems such as Wiki based document management system, VCS and ITS as a result of developing software product in an education setting can present useful insights on runtime development process and process quality to the course instructor. We conduct a series of experiments which reveal that not only product but process quality varies significantly between student teams and mining process aspects can help the instructor in giving directed and specific feedback. We observe commit patterns characterizing equal and un-equal distribution of workload between team members, patterns indicating consistent activity in contrast to spike in activity just before the deadline, varying quality of commit messages, developer and component entropy, variation in degree of process compliance and bug fixing quality. While the metrics and visualization proposed are useful for the instructor, they should be handled with care when used for grading, because they can be easily tricked by redundant or needless commits or issues (bug reports). We believe the proposed framework is effective in providing visibility to the instructor on process data which can be mined to derive actionable information for improving academic outcome and improve the teaching and learning methodology.

8. REFERENCES

- [1] Chiara Francalanci and Francesco Merlo. Empirical analysis of the bug fixing process in open source projects. In *Open Source Development, Communities and Quality*, pages 187–196. Springer, 2008.
- [2] Louis Glassy. Using version control to observe student software development processes. *J. Comput. Sci. Coll.*, 21(3):99–106, February 2006.
- [3] James M. Hogan and Richard Thomas. Developing the software engineering team. In *Proceedings of the 7th Australasian conference on Computing education - Volume 42*, ACE ’05, pages 203–210, Darlinghurst, Australia, Australia, 2005. Australian Computer Society, Inc.
- [4] Curt Jones. Using subversion as an aid in evaluating individuals working on a group coding project. *J. Comput. Sci. Coll.*, 25(3):18–23, January 2010.
- [5] Judy Kay, Nicolas Maisonneuve, Kalina Yacef, and Osmar Zaiane. Mining patterns of events in students teamwork data. In *Educational Data Mining Workshop and Intelligent Tutoring Systems*, pages 1–8, 2006.
- [6] Foutse Khomh, Brian Chan, Ying Zou, and Ahmed E. Hassan. An entropy evaluation approach for triaging field crashes: A case study of mozilla firefox. In *Proceedings of the 2011 18th Working Conference on Reverse Engineering*, WCRE ’11, pages 261–270, Washington, DC, USA, 2011. IEEE Computer Society.
- [7] Sangeeta Lal and Ashish Sureka. Comparison of seven bug report types: A case-study of google chrome browser project. In *APSEC*, pages 517–526, 2012.
- [8] Y. Liu, E. Stroulia, K. Wong, and D. German. Using CVS historical information to understand how students develop software. In *MRS 2004: International Workshop on Mining Software Repositories*, 2004.
- [9] Keir Mierle, Kevin Laven, Sam Roweis, and Greg Wilson. Mining student CVS repositories for performance indicators. In *Proceedings of the 2005 international workshop on Mining software repositories*, MSR ’05, pages 1–5, 2005.
- [10] Johan Peeters. Agile security requirements engineering. In *Symposium on Requirements Engineering for Information Security*, 2005.
- [11] Wouter Poncin, Alexander Serebrenik, and Mark van den Brand. Mining student capstone projects with FRASR and ProM. In *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications companion*, SPLASH ’11, pages 87–96, 2011.
- [12] Michael J Rees. A feasible user story tool for agile software development? In *Software Engineering Conference, 2002. Ninth Asia-Pacific*, pages 22–30. IEEE, 2002.
- [13] T. Reichlmayr. Enhancing the student project team experience with blended learning techniques. In *Frontiers in Education, 2005. FIE ’05. Proceedings 35th Annual Conference*, 2005.
- [14] G. Robles and J.M. Gonzalez-Barahona. Mining student repositories to gain learning analytics. an experience report. In *Global Engineering Education Conference (EDUCON), 2013 IEEE*, pages 1249–1254, 2013.
- [15] Wil M. P. van der Aalst. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer Publishing Company, Incorporated, 1st edition, 2011.

⁷[http:// fluxicon.com/disco/files/Disco-Tour.pdf](http://fluxicon.com/disco/files/Disco-Tour.pdf)